AI agents turn our fatigue tests into a lean openBIS data structure

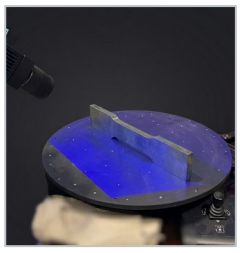
[Authors] · BAM, 7.2 Buildings and Structures

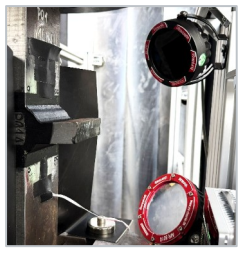
BAM
Bundesanstalt für Materialforschung und -prüfung

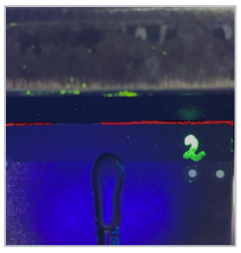
Idea Describe the experiment once, in domain language. AI agents check it against the BAM data standard and the live BAM Data Store, design a flat structure and create only what is really missing. Humans approve every decision.

1 The tests: S-N curves of welded joints

Butt and cruciform joints, randomised load levels – scatter starts before the test, so every specimen must be traceable.


3D scan


test rig


crack monitoring


fracture surface

01–02 Preparation — quality check, grinding
03–06 Characterisation — 3D scan, weld rating, geometry
07–11 Test rig — gauges, cyclic test, evaluation
12 Post-test — fracture surface

Per specimen: 34 data items · 10 instruments · 17 file kinds

2 The agent approach: check first

Workflow 12 steps, domain language
BAM standard bam-masterdata, 141 object types
BAM Data Store what exists? read-only

AI agents + scripts
understand · design · compare – every element gets one verdict

20 REUSE types, fields, file types used as is
2 EXTEND existing lists get new terms
41 CREATE 2 object types, 6 lists, 33 fields

Humans approve twice (see 4). Only missing elements are created; new types go to the BAM standard as a pull request.

3 Seven steps: scripts count, agents decide, humans approve

1 Catalog the standard → 2 Inventory the instance → 3 Read the workflow → 4 Design a flat structure → 5 Gap analysis, 3 verdicts → 6 Python classes, import-tested → 7 Apply to openBIS, dry run first

■ AI agent ■ deterministic script

✔ human approval (after 4) ✔ human approval (after 5)

4 What the human checks

✔ Checkpoint 1 · Design design.md – every step gets one place

01 Quality check	specimen, section 01	6 fields
03 3D scan	own SCAN object	GOM scanner, raw data
10 Fatigue test	TEST, section 10	machine, load programme

34 data items + 17 file kinds placed · 0 unplaced · 8 questions answered

✔ Checkpoint 2 · Gap report gap-report.md – one verdict per element

REUSE	testing machine	standard, object_types.py:1604
EXTEND	weld type list	+2 terms, vocabularies.py:38
CREATE	cyclic fatigue test	nearest: FCG test – no S-N data

file:line in the standard or a reason – nothing is created unchecked

5 Result: one specimen, three steps, one level

Specimen "Next step" on top · sections 01 · 02 · 04 · 05 · 06

SCAN steps 03–04
TEST steps 07–11
FRACTURE step 12

Instruments & consumables: created once in the inventory, linked as parents

2 new object types
6+2 vocabularies new + extended
0 new dataset types
1 link depth

Take-home

Check, don't just generate deterministic catalog, real import test, dry run
Humans decide design & gap report approved; domain calls stay with experts
Reusable pattern any department: describe → agents check → approve

Sicherheit in Technik und Chemie